

```

1 #include <stdio.h>
2 #include <stdint.h>
3
4 // vector_sum: takes two vectors represented as array of double and adds them
5 // together by adding the components at each index. Return a new array
6 // vector_sum({ 1.2, 3.4 }, {-1.0, 3.6 }) => { 0.2, 7.0 }
7 // Assume the vectors have the same length
8
9 /* Q: What happens if double[] is used as a return type?
10
11 double[] vector_sum(double vec1[], double vec2[]);
12 */
13
14 /* We can use double* as a return type though!
15    Q: What problem will we have with this version?
16
17 double* vector_sum(double vec1[], double vec2[]) {
18     double result[...];
19 }
20 */
21
22 // Pass in length as an argument. Maybe now we've got it!
23 double* vector_sum(double vec1[], double vec2[], int len) {
24     double result[len];
25     printf("vec1:\t%p\nvec2:\t%p\nresult:\t%p\n", vec1, vec2, result);
26     for(int i = 0; i < len; i += 1) {
27         result[i] = vec1[i] + vec2[i];
28     }
29     return result;
30 }
31
32 int main() {
33     double v1[] = { 1.3, 4.2 };
34     double v2[] = { 1.5, -1 };
35     double* v3 = vector_sum(v1, v2, 2);
36
37     double v4[] = { 100, 100 };
38     double* v5 = vector_sum(v4, v3, 2);
39
40     printf("first element of v3, v5: %f %f\n", v3[0], v5[0]);
41
42     printf("v1: %p\n", v1);
43     printf("v2: %p\n", v2);
44     printf("v3: %p\n", v3);
45     printf("v4: %p\n", v4);
46     printf("v5: %p\n", v5);
47
48 }

```

Variable/Role

Address

Data

0/8 1/9 2/A 3/B 4/C 5/D 6/E 7/F

0x...88

0x...90

0x...98

0x...A0

0x...A8

0x...B0

0x...B8

0x...C0

0x...C8

0x...D0

0x...D8

0x...E0

0x...E8

0x...F0

0x...F8

0x...00

0x...08

0x...10

0x...18

0x...20

0x...28

0x...30

0x...38

0x...40

0x...48

0x...50

0x...58

0x...60

0x...68

0x...70

0x...78

0x...80

```

$ gcc concat.c -o concat
concat.c:29:10: warning: address of stack
memory associated with local variable
'result' returned [-Wreturn-stack-address]
   29 |     return result;
      |     ~~~~~
$ ./concat
vec1: 0x16d0df010
vec2: 0x16d0df000
result: 0x16d0def40
vec1: 0x16d0def40
vec2: 0x16d0def40
result: 0x16d0def40
first element of v3, v5: 100.000000 100.000000
v1: 0x16d0df010
v2: 0x16d0df000
v3: 0x16d0def40
v4: 0x16d0def40
v5: 0x16d0def40

```

```
1 #include <stdio.h>
2 #include <stdint.h>
3
4 void wheresmystuff(char* s) {
5     int x = 12;
6     uint8_t y = 53;
7     char z = 9;
8     double ns[] = { 4.0, 3.0, 9.0 };
9
10    printf("x=%d, %lu bytes, starts at: %p\n", x, sizeof(x), &x);
11    printf("y=%hhu, %lu bytes, starts at: %p\n", y, sizeof(y), &y);
12    printf("z=%hhd, %lu bytes, starts at: %p\n", z, sizeof(z), &z);
13    printf("ns=[%f,%f,%f], %lu bytes, starts at: %p\n",
14          ns[0], ns[1], ns[2], sizeof(ns), &ns);
15
16    printf("s=\"%s\"@%p, %lu bytes, starts at: %p\n", s, s, sizeof(s), &s);
17 }
18
19 int main() {
20     char str[] = "14 char string";
21     wheresmystuff(str);
22     printf("\nstr takes up %lu bytes starting at: %p\n", sizeof(str), &str);
23 }
```

```
$ # NOTE - Joe ran this on his Macbook to get this output
$ gcc wheresmystuff.c -o wheresmystuff
$ ./wheresmystuff
x=12, 4 bytes, starts at: 0x16f4c6e44
y=53, 1 bytes, starts at: 0x16f4c6e43
z=9, 1 bytes, starts at: 0x16f4c6e42
ns=[4.000000,3.000000,9.000000], 24 bytes, starts at: 0x16f4c6e50
s="14 char string"@0x16f4c6e98, 8 bytes, starts at: 0x16f4c6e48

str takes up 15 bytes starting at: 0x16f4c6e98
```

Variable/Role	Address	Data						
		0/8	1/9	2/A	3/B	4/C	5/D	6/E 7/F
	0x...00							
	0x...08							
	0x...10							
	0x...18							
	0x...20							
	0x...28							
	0x...30							
	0x...38							
	0x...40							
	0x...48							
	0x...50							
	0x...58							
	0x...60							
	0x...68							
	0x...70							
	0x...78							
	0x...80							
	0x...88							
	0x...90							
	0x...98							
	0x...A0							
	0x...A8							
	0x...B0							
	0x...B8							
	0x...C0							
	0x...C8							
	0x...D0							
	0x...D8							
	0x...E0							
	0x...E8							
	0x...F0							
	0x...F8							